

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-a34e3beedf921fadf.jsonl`  
- SHA-256: `9d08e06435647e167094dc8d055b89023180e25de35f182831f5ccb24caef688`  
- Source modified: `2026-07-06T09:13:52+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_3512746c-54e`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T09:12:51.504Z)

Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-selective-scan -> main, one commit db87cc0).
It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels, /watchlist commands +
an ingestion gate in telethon_client.py that skips unwatched sources. Changed files ONLY:
command_service.py (new /watch etc + _resolve_source), commands.py (re-export patterns), db.py (set_source_watched, list_account_sources), telethon_client.py (the gate),
tests/test_command_service.py (10 tests).
Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):

- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear in /channels for opt-in;
/channels should also upsert candidates from client.get_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db._apply_schema on aiosqlite ":memory:"); seed via db.upsert_account + db.upsert_source; drive command_service.CommandService().handle()).
NOTE: real telegram sources have a NUMERIC provider_source_id (str(channel_id) e.g. '-1001234567890'); upsert_channel
creates them with watched=True. The on_new_message gate is inside a '# pragma: no cover' function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest 492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value finding. No praise.
Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-contract violation users hit),
medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code AND docs/adr/0009-selective-channel-scanning.md, trace real control flow, run a repro with

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the defect genuinely ships in THIS PR's diff
 and (for ADR-conformance) actually deviates from the accepted ADR. REFUTED if the code conforms / trigger can't occur /
 pre-existing. PLAUSIBLE if unsettled. Corrected severity.
 FINDING (commands): lstrip('@') strips multiple leading @ characters, accepting malformed input as valid
 severity low | src/tg_video_filter/command_service.py:417
 summary: _resolve_source uses target.lstrip('@'), which strips ALL leading '@' rather than a single prefix, so '/watch @@@-1001234567890' resolves the same as '/watch -1001234567890'.
 scenario: User types '/watch @@@-1001234567890' (typo / fat-finger). Instead of an error, the command silently succeeds and starts watching the channel. Reproduced: '/watch @@@-1001234567890' -> '? Now watching **Classical Music Daily**'. Latent/cosmetic ? no data loss, but it accepts clearly-malformed identifiers as valid, which can mask user error and complicates future username handling.
 evidence: command_service.py:417 target = target.lstrip('@'). str.lstrip treats its argument as a set of characters to remove repeatedly, not a prefix. Confirmed in the repro output above.
 rec: Strip a single leading '@' only, e.g. target = target[1:] if target.startswith('@') else target, so multi-@ inputs are treated as not-found / invalid rather than silently normalized.

2. assistant (2026-07-06T09:12:55.035Z)

I'll verify this finding by reading the actual code and tracing control flow.

3. user (2026-07-06T09:12:56.018Z)

```

395         )
396         lines = ["? **Watched sources**", ""]
397         for s in sources:
398             label = s.title or s.provider_source_id
399             lines.append(f"? [{s.provider_source_id}] ? {label}")
400         return CommandResult("\n".join(lines))
401
402     async def _resolve_source(
403         self, conn: aioredis.Connection, user_id: int, target: str
404     ) -> Source | None:
405         """Resolve a user-provided source identifier to a ``Source`` row.
406
407         Accepts ``@username`` or a numeric channel id string. The ``@`` prefix
408         is stripped before lookup since Telegram channel ids are numeric.
409         """
410         target = target.lstrip("@")
411         # Try by provider-native id first (the common case).
412         src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
413         if src is not None:
414             return src
415         # Fall back: try with @ prefix in the target column (just in case).
416         return await db.get_source_by_provider_id(conn, user_id, "telegram",
417 f"@{target}")
418
419     # -- internal -----
420
421     async def _update(
422         self,
423         conn: aioredis.Connection,
424         user_id: int,
425         current: FilterConfig,
426         max_size_mb: int | None = None,
427         max_duration_s: int | None = None,
428         min_duration_s: int | None = None,
429         target_height: int | None = None,
430         exclude_gif: bool | None = None,
431         exclude_round: bool | None = None,
432         exclude_nosound: bool | None = None,
433     ) -> FilterConfig:

```

```

433         """Apply partial updates to the account's config, persist, and return."""
434         updates: dict[str, object] = {}
435         if max_size_mb is not None:
436             updates["max_size_mb"] = max_size_mb
437         if max_duration_s is not None:
438             updates["max_duration_s"] = max_duration_s
439         if min_duration_s is not None:
440             updates["min_duration_s"] = min_duration_s
441         if target_height is not None:
442             updates["target_height"] = target_height
443         if exclude_gif is not None:
444             updates["exclude_gif"] = exclude_gif

```

4. assistant (2026-07-06T09:12:58.491Z)

The finding is about ``rstrip("@")``. Let me verify the behavior and reproduce it. First let me check the seeding helpers and confirm the repro.

5. user (2026-07-06T09:13:00.622Z)

```

-1001234567890
name

```

6. assistant (2026-07-06T09:13:04.959Z)

``rstrip("@")`` does strip all leading ``@``, confirmed. Now let me run the actual repro through the command service to confirm ``/watch @@@-1001234567890`` succeeds.

7. user (2026-07-06T09:13:06.466Z)

```

199:async def _apply_schema(conn: aiosqlite.Connection) -> None:
936:async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
1104:async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
1142:async def get_source_by_provider_id(
1537:async def set_source_watched(

```

8. user (2026-07-06T09:13:09.994Z)

```

1104     async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
1105         """Insert a source row, or update title/watched on conflict.
1106
1107         The conflict key is ``(account_id, provider, provider_source_id)`` per the
1108         ADR 0011 schema. Returns the persisted row.
1109         """
1110         await conn.execute(
1111             """INSERT INTO sources
1112                 (account_id, provider, provider_source_id, title, kind, watched,
1113                 seen_first_at, last_seen_at)
1114                 VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1115                 ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
1116                     title=excluded.title,
1117                     watched=excluded.watched,
1118                     last_seen_at=excluded.last_seen_at""",
1119             (
1120                 source.account_id,
1121                 source.provider,
1122                 source.provider_source_id,
1123                 source.title,
1124                 source.kind,
1125                 int(source.watched),
1126                 source.seen_first_at.isoformat(),
1127                 (source.last_seen_at.isoformat() if source.last_seen_at else None),
1128             ),
1129         )

```

```

1130     await conn.commit()
1131     conn.row_factory = aiosqlite.Row
1132     async with conn.execute(
1133         """SELECT * FROM sources
1134             WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1135         (source.account_id, source.provider, source.provider_source_id),
1136     ) as cur:
1137         row = await cur.fetchone()
1138     assert row is not None # just upserted by the conflict key
1139     return _row_to_source(row)
1140
1141
1142     async def get_source_by_provider_id(
1143         conn: aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
1144     ) -> Source | None:
1145         """Resolve a source by its provider-native id."""
1146         conn.row_factory = aiosqlite.Row
1147         async with conn.execute(
1148             """SELECT * FROM sources
1149                 WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1150             (account_id, provider, provider_source_id),
1151         ) as cur:
1152             row = await cur.fetchone()
1153         return _row_to_source(row) if row else None
1154
1155
1156     # --- ContentItem repos (ADR 0011) -----
1157
1158
1159     async def insert_content_item(conn: aiosqlite.Connection, item: ContentItem) ->
ContentItem:
1160         """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
1161
1162         The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
1163         unique constraints guard against duplicates:
1164         - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
1165         - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
1166         """
1167         cur = await conn.execute(
1168             """INSERT INTO content_items
1169                 (account_id, source_id, provider, provider_message_id, content_type,
1170                 text, link_url, media_ref, document_id, duration_s, width, height,
1171                 size_bytes, mime_type, dedup_key, matched, seen_at)
1172                 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
1173             (

```

9. assistant (2026-07-06T09:13:12.417Z)

Now let me look at the models and the handle/watch flow to build a repro.

10. user (2026-07-06T09:13:14.136Z)

```

35:class Account(BaseModel):
123:class Source(BaseModel):
127:    providers add their own source kinds). ``provider_source_id`` is the
136:    provider_source_id: str = Field(

```

11. assistant (2026-07-06T09:13:16.631Z)

Rather than build a full repro from scratch, let me look at how the tests seed data, then reuse that pattern.

12. user (2026-07-06T09:13:16.930Z)

```

1      """Tests for the provider-independent CommandService (command_service.py).
2
3      These tests exercise command parsing and config mutation without any
4      Telethon objects ? pure strings in, CommandResult out. The Telethon
5      adapter (commands.py) is tested separately via test_commands.py.
6      """
7
8      from __future__ import annotations
9
10     import aiosqlite
11     import pytest
12
13     from tg_video_filter import db
14     from tg_video_filter.command_service import (
15         CommandService,
16         format_commands,
17         parse_duration_s,
18         parse_size_mb,
19     )
20     from tg_video_filter.models import FilterConfig, Source
21
22     USER_ID = 111
23
24
25     @pytest.fixture
26     async def conn() -> aiosqlite.Connection:
27         """An in-memory SQLite connection with a seeded user and default config."""
28         c = await aiosqlite.connect(":memory:")
29         await db._apply_schema(c)
30         await db.upsert_user(
31             c,
32             db.User(id=USER_ID, phone="+15555550100"), # type: ignore[attr-defined]
33         )
34         await db.save_filter_config(c, USER_ID, FilterConfig())
35         yield c
36         await c.close()
37
38
39     @pytest.fixture
40     def svc() -> CommandService:
41         return CommandService()
42
43
44     # --- unit parser tests -----
45
46
47     @pytest.mark.parametrize(
48         "value, unit, expected",
49         [
50             ("10", "MB", 10),
51             ("1", "GB", 1024),
52             ("500", "KB", 1),
53             ("5", "", 5),
54             ("2", "TB", 2 * 1024 * 1024),
55         ],
56     )
57     def test_parse_size_mb(value: str, unit: str, expected: int) -> None:
58         assert parse_size_mb(value, unit) == expected
59
60
61     def test_parse_size_mb_unknown_unit_raises() -> None:
62         with pytest.raises(ValueError):
63             parse_size_mb("10", "XB")
64
65

```

```

66     @pytest.mark.parametrize(
67         "value, unit, expected",
68         [
69             ("15", "m", 900),
70             ("2", "h", 7200),
71             ("90", "s", 90),
72             ("5", "", 5),
73         ],
74     )
75     def test_parse_duration_s(value: str, unit: str, expected: int) -> None:
76         assert parse_duration_s(value, unit) == expected
77
78
79     def test_parse_duration_s_unknown_unit_raises() -> None:
80         with pytest.raises(ValueError):
81             parse_duration_s("10", "x")
82
83
84     # --- /set size -----
85
86
87     async def test_set_size(conn: aiomysqlite.Connection, svc: CommandService) -> None:
88         result = await svc.handle(USER_ID, "/set size 100MB", conn, FilterConfig())
89         assert "100MB" in result.reply_text
90         assert result.config_changed is True
91         loaded = await db.load_filter_config(conn, USER_ID)
92         assert loaded.max_size_mb == 100
93
94
95     async def test_set_size_regex_no_match_returns_empty(
96         conn: aiomysqlite.Connection, svc: CommandService
97     ) -> None:
98         """Input that doesn't match the _SIZE_RE regex returns empty (noise path).
99
100         The regex enforces valid units ? this isn't testing ValueError from
101         parse_size_mb (that branch is unreachable). See review finding #3.
102         """
103         result = await svc.handle(USER_ID, "/set size not-a-number", conn, FilterConfig())
104         assert result.reply_text == ""
105         assert result.config_changed is False
106
107
108     async def test_set_size_below_minimum(conn: aiomysqlite.Connection, svc: CommandService)
-> None:
109         result = await svc.handle(USER_ID, "/set size 0MB", conn, FilterConfig())
110         assert "at least 1 MB" in result.reply_text
111         assert result.config_changed is False
112
113
114     # --- /set length -----
115
116
117     async def test_set_length(conn: aiomysqlite.Connection, svc: CommandService) -> None:
118         result = await svc.handle(USER_ID, "/set length 15m", conn, FilterConfig())
119         assert "15m" in result.reply_text or "900s" in result.reply_text
120         assert result.config_changed is True
121         loaded = await db.load_filter_config(conn, USER_ID)
122         assert loaded.max_duration_s == 900
123
124
125     async def test_set_length_below_min(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
126         cfg = FilterConfig(min_duration_s=60)
127         result = await svc.handle(USER_ID, "/set length 30s", conn, cfg)
128         assert "must be >= min duration" in result.reply_text

```

```

129         assert result.config_changed is False
130
131
132     # --- /set min length -----
133
134
135     async def test_set_min_length(conn: aiomysql.Connection, svc: CommandService) -> None:
136         result = await svc.handle(USER_ID, "/set min length 5s", conn, FilterConfig())
137         assert "5s" in result.reply_text
138         assert result.config_changed is True
139         loaded = await db.load_filter_config(conn, USER_ID)
140         assert loaded.min_duration_s == 5
141
142
143     async def test_set_min_length_above_max(conn: aiomysql.Connection, svc: CommandService)
-> None:
144         cfg = FilterConfig(max_duration_s=10)
145         result = await svc.handle(USER_ID, "/set min length 60s", conn, cfg)
146         assert "must be <= max duration" in result.reply_text
147         assert result.config_changed is False
148
149
150     # --- /set min resolution -----
151
152
153     async def test_set_min_resolution(conn: aiomysql.Connection, svc: CommandService) ->
None:
154         result = await svc.handle(USER_ID, "/set min resolution 720p", conn, FilterConfig())
155         assert "720p" in result.reply_text
156         assert result.config_changed is True
157         loaded = await db.load_filter_config(conn, USER_ID)
158         assert loaded.target_height == 720
159
160
161     async def test_set_min_resolution_below_min(
162         conn: aiomysql.Connection, svc: CommandService
163     ) -> None:
164         result = await svc.handle(USER_ID, "/set min resolution 0p", conn, FilterConfig())
165         assert "at least 1p" in result.reply_text
166         assert result.config_changed is False
167
168
169     async def test_set_min_res_help_on_malformed(
170         conn: aiomysql.Connection, svc: CommandService
171     ) -> None:
172         result = await svc.handle(USER_ID, "/set min resolution abc", conn, FilterConfig())
173         assert "Usage:" in result.reply_text
174
175
176     # --- /set exclude -----
177
178
179     async def test_set_exclude_gif_on(conn: aiomysql.Connection, svc: CommandService) ->
None:
180         result = await svc.handle(USER_ID, "/set exclude gif on", conn, FilterConfig())
181         assert "ON" in result.reply_text
182         assert result.config_changed is True
183         loaded = await db.load_filter_config(conn, USER_ID)
184         assert loaded.exclude_gif is True # verify correct field mutated
185
186
187     async def test_set_exclude_round_off(conn: aiomysql.Connection, svc: CommandService) ->
None:
188         result = await svc.handle(USER_ID, "/set exclude round off", conn, FilterConfig())
189         assert "OFF" in result.reply_text

```

```

190         assert result.config_changed is True
191
192
193     async def test_set_exclude_help_on_malformed(
194         conn: aiomysqlite.Connection, svc: CommandService
195     ) -> None:
196         result = await svc.handle(USER_ID, "/set exclude gif true", conn, FilterConfig())
197         assert "Usage:" in result.reply_text
198         assert result.config_changed is False
199
200
201     # --- /show config -----
202
203
204     async def test_show_config(conn: aiomysqlite.Connection, svc: CommandService) -> None:
205         result = await svc.handle(USER_ID, "/show config", conn, FilterConfig())
206         assert "Current Filter Configuration" in result.reply_text
207         assert result.config_changed is False
208
209
210     # --- /show commands -----
211
212
213     async def test_show_commands(conn: aiomysqlite.Connection, svc: CommandService) -> None:
214         result = await svc.handle(USER_ID, "/show commands", conn, FilterConfig())
215         assert "Available Commands" in result.reply_text
216         assert result.config_changed is False
217
218
219     # --- unrecognized command -----
220
221
222     async def test_unrecognized_returns_empty(conn: aiomysqlite.Connection, svc:
CommandService) -> None:
223         result = await svc.handle(USER_ID, "hello world", conn, FilterConfig())
224         assert result.reply_text == ""
225         assert result.config_changed is False
226
227
228     # --- format_commands -----
229
230
231     def test_format_commands_includes_all_sections() -> None:
232         text = format_commands()
233         assert "/show config" in text
234         assert "/set size" in text
235         assert "/set exclude" in text
236         assert "/backfill" in text
237
238
239     # --- /channels + /watch + /unwatch + /watchlist (ADR 0009, Track E) -----
240
241
242     async def _seed_source(
243         conn: aiomysqlite.Connection,
244         account_id: int,
245         provider_source_id: str,
246         title: str = "",
247         watched: bool = False,
248     ) -> Source:
249         return await db.upsert_source(
250             conn,
251             Source(
252                 account_id=account_id,
253                 provider="telegram",

```



```

254         provider_source_id=provider_source_id,
255         title=title,
256         kind="channel",
257         watched=watched,
258     ),
259 )
260
261
262 async def test_channels_lists_all_sources_with_watch_status(
263     conn: aiomysqlite.Connection, svc: CommandService
264 ) -> None:
265     await _seed_source(conn, USER_ID, "-1001", "Channel A", watched=True)
266     await _seed_source(conn, USER_ID, "-1002", "Channel B", watched=False)
267     result = await svc.handle(USER_ID, "/channels", conn, FilterConfig())
268     assert "Channel A" in result.reply_text
269     assert "Channel B" in result.reply_text
270     assert "?" in result.reply_text # watched indicator
271     assert "?" in result.reply_text # unwatched indicator
272
273
274 async def test_channels_empty_shows_help(conn: aiomysqlite.Connection, svc:
CommandService) -> None:
275     result = await svc.handle(USER_ID, "/channels", conn, FilterConfig())
276     assert "No sources yet" in result.reply_text
277
278
279 async def test_watchlist_only_watched(conn: aiomysqlite.Connection, svc: CommandService)
-> None:
280     await _seed_source(conn, USER_ID, "-1001", "Ch A", watched=True)
281     await _seed_source(conn, USER_ID, "-1002", "Ch B", watched=False)
282     result = await svc.handle(USER_ID, "/watchlist", conn, FilterConfig())
283     assert "Ch A" in result.reply_text
284     assert "Ch B" not in result.reply_text
285
286
287 async def test_watchlist_empty_shows_help(conn: aiomysqlite.Connection, svc:
CommandService) -> None:
288     result = await svc.handle(USER_ID, "/watchlist", conn, FilterConfig())
289     assert "No watched sources" in result.reply_text
290
291
292 async def test_watch_by_numeric_id(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
293     await _seed_source(conn, USER_ID, "-1001", "Target", watched=False)
294     result = await svc.handle(USER_ID, "/watch -1001", conn, FilterConfig())
295     assert "Now watching" in result.reply_text
296     src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1001")
297     assert src is not None
298     assert src.watched is True
299
300
301 async def test_watch_by_username(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
302     await _seed_source(conn, USER_ID, "@mychan", "My Channel", watched=False)
303     result = await svc.handle(USER_ID, "/watch @mychan", conn, FilterConfig())
304     assert "Now watching" in result.reply_text
305     src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "@mychan")
306     assert src is not None
307     assert src.watched is True
308
309
310 async def test_unwatch(conn: aiomysqlite.Connection, svc: CommandService) -> None:
311     await _seed_source(conn, USER_ID, "-1001", "Target", watched=True)
312     result = await svc.handle(USER_ID, "/unwatch -1001", conn, FilterConfig())
313     assert "Stopped watching" in result.reply_text

```

```

314         src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1001")
315         assert src is not None
316         assert src.watched is False
317
318
319     async def test_watch_unknown_source_shows_error(
320         conn:aiosqlite.Connection, svc: CommandService
321     ) -> None:
322         result = await svc.handle(USER_ID, "/watch -9999", conn, FilterConfig())
323         assert "not found" in result.reply_text
324
325
326     async def test_set_source_watched_toggle(conn:aiosqlite.Connection) -> None:
327         src = await _seed_source(conn, USER_ID, "-2000", watched=True)
328         assert src.id is not None
329         await db.set_source_watched(conn, USER_ID, src.id, False)
330         reloaded = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-2000")
331         assert reloaded is not None
332         assert reloaded.watched is False
333
334
335     async def test_list_account_sources_watched_only(
336         conn:aiosqlite.Connection,
337     ) -> None:
338         await _seed_source(conn, USER_ID, "-10", watched=True)
339         await _seed_source(conn, USER_ID, "-20", watched=False)
340         all_srcs = await db.list_account_sources(conn, USER_ID)
341         assert len(all_srcs) == 2
342         watched = await db.list_account_sources(conn, USER_ID, watched_only=True)
343         assert len(watched) == 1
344         assert watched[0].provider_source_id == "-10"
345

```

13. assistant (2026-07-06T09:13:20.971Z)

Now let me write a repro that seeds a source with a numeric id and calls `/watch @@@-1001234567890`.

14. user (2026-07-06T09:13:28.728Z)

```

'/watch @@@-1001234567890' -> '? Now watching **Classical Music Daily** (`-1001234567890`).'
'/unwatch @@@-1001234567890' -> '? Stopped watching **Classical Music Daily**
(`-1001234567890`).'

```

15. assistant (2026-07-06T09:13:33.791Z)

The repro confirms the behavior exactly as the finding describes: `/watch @@@-1001234567890` succeeds and starts watching the channel, when a strict single-prefix strip would treat it as not-found.

Now let me assess severity honestly and whether this is a genuine defect in the PR's diff.

The behavior is confirmed and it is introduced by this PR (`\_resolve\_source` is new in `command\_service.py`, line 410 in the actual file ? the finding cites 417, close enough; the exact `rstrip("@")` line is at 410). Let me confirm this line is in the diff and not pre-existing.

16. user (2026-07-06T09:13:34.681Z)

```

src\tg_video_filter\command_service.py:345:         src = await self._resolve_source(conn,
user_id, target)
src\tg_video_filter\command_service.py:361:         src = await self._resolve_source(conn,
user_id, target)
src\tg_video_filter\command_service.py:402:     async def _resolve_source(

```

```
src\tg_video_filter\command_service.py:410:         target = target.lstrip("@")
```

17. assistant (2026-07-06T09:13:45.174Z)

Confirmed: `\_resolve\_source` and the `lstrip("@)` are new code in this PR's diff (Track E). The finding is factually correct and reproducible.

Assessment of severity:

- No data loss, no broken feature on the live path. The malformed input `@@@-1001234567890` resolves to a real, existing source and watches it ? which is arguably what the user intended anyway (they wanted to watch that channel; the extra `@`'s are a typo).
- ADR 0009 line 64 says `/watch <#name|id>`. It does not specify strict `@`-prefix parsing. Nothing in the ADR is violated.
- The practical impact is that clearly-malformed input is silently normalized instead of rejected. This is genuinely cosmetic/hygiene: it can only ever *succeed* against a source that actually exists under the stripped id; against a non-existent id it still returns "not found". So it masks user typos rather than producing wrong behavior.

The finding is real but the impact is minimal and lands squarely at low severity. The finding's own severity (low) is accurate.

18. user (2026-07-06T09:13:52.364Z)

Structured output provided successfully